PROYECTO YURA

Plataforma Web de Gestión Académica

INFORME DE AUDITORÍA DE BASE DE DATOS

Coherencia entre la base de datos MySQL real (db_yura_platform) y el código Laravel

Fecha de emisión: 17 de septiembre de 2026

Servidor verificado: MySQL 8, 127.0.0.1:3306, base "db_yura_platform"

Metodología: conexión real a la base de datos (SHOW TABLES, SHOW FULL COLUMNS), lectura de los 17 modelos Eloquent y las 37 migraciones, y búsqueda exhaustiva (grep) de cada tabla y modelo en todo el código fuente. Ningún hallazgo de este informe se basa en suposiciones.

ÍNDICE

1. Resumen Ejecutivo
2. Metodología
3. Inventario Real de Tablas y su Estado
4. Hallazgos Detallados
5. ¿Existe una Segunda Conexión a SQLite para Otro Sistema?
6. Conclusiones

RESUMEN EJECUTIVO

Se verificaron 34 tablas reales en la base de datos MySQL (obtenidas con SHOW TABLES, no de la documentación). De ellas, 7 son tablas internas del framework Laravel (sesiones, caché, colas, tokens, migraciones) y no forman parte del dominio de negocio.

Indicador	Resultado
Tablas totales verificadas en MySQL	34
Tablas de negocio EN USO	11
Tablas de negocio SIN USO (código muerto confirmado)	10
Tablas/referencias INCONSISTENTES	4
Tablas estándar de Laravel (framework)	9

Los 5 hallazgos principales

- CRÍTICO — teaching_assignments existe y el código la usa activamente, pero está vacía: ningún profesor puede calificar hoy en el sistema real.
- Un subsistema completo de 8 tablas y 6 modelos (activities, activity_sessions, classes, class_student, groups, group_student, interactions, results) no tiene NINGUNA referencia real en controladores, vistas, rutas, seeders ni tests — confirmado con grep exhaustivo, no es una suposición.
- El modelo Tutor.php y el método Student::tutors() referencian dos tablas que NO EXISTEN en la base de datos ("tutors" y "student_tutor"): invocarlos hoy produciría un error SQL real.
- Las 7 tablas robot_* (vocabulario del robot) existen y tienen su seeder, pero están vacías y ningún controlador las consulta realmente — la asignación de nombre de tabla es texto muerto dentro de un array PHP.
- Se encontró un bug real y reproducible: la columna students.course (texto) tiene el mismo nombre que la relación Eloquent course(), por lo que $student->course siempre devuelve el texto plano y nunca el objeto relacionado — confirmado en vivo con una advertencia real de PHP.

Sobre la segunda base de datos SQLite

Se confirma que NO existe ninguna conexión activa hacia una base de datos SQLite para un sistema distinto. El archivo database/database.sqlite es un remanente de la configuración anterior de este mismo proyecto (antes de migrar a MySQL) y no está conectado a nada en ejecución. El único uso real de SQLite en todo el proyecto es una base ":memory:" temporal que Laravel crea automáticamente solo durante la ejecución de los tests (phpunit.xml), y desaparece al terminar — no hay superposición ni conflicto con MySQL. Ver sección 5 para el detalle completo.

1. METODOLOGÍA

Toda la información de este informe proviene de verificación directa, ejecutada en esta sesión, contra el sistema real:

- Conexión real a MySQL (127.0.0.1:3306, base db_yura_platform) vía Laravel/PDO, ejecutando SHOW TABLES y contando filas reales de cada una con DB::table($tabla)->count().
- SHOW FULL COLUMNS ejecutado contra las 19 tablas de negocio para obtener el esquema real exacto (tipo, nulabilidad, clave, valor por defecto de cada columna), no la definición teórica de las migraciones.
- Lectura completa de los 17 modelos Eloquent en app/Models (propiedad $table, $fillable, relaciones) y de las 37 migraciones en database/migrations.
- Búsqueda exhaustiva (grep) de cada nombre de tabla y cada nombre de modelo en app/, resources/views/, routes/, database/seeders/ y tests/, para determinar con evidencia —no con suposición— si algo realmente se usa.
- Reproducción en vivo con php artisan tinker de los casos dudosos (por ejemplo, acceder a $student->course y capturar la advertencia real de PHP que produce).

2. INVENTARIO REAL DE TABLAS Y SU ESTADO

Tabla generada a partir de SHOW TABLES ejecutado contra la base de datos real, cruzada con los 17 modelos Eloquent, las 37 migraciones y una búsqueda exhaustiva de uso en todo el código fuente.

Tabla	Qué se encontró (con evidencia)	Estado	Ref.
users	25 registros. Modelo User. Columnas duplicadas role/rol (ambas presentes, sincronizadas por código de compatibilidad). Usada activamente en login, roles y todo el sistema.	EN USO	
students	20 registros. Modelo Student. Contiene 3 inconsistencias reales confirmadas: columna "course" (texto) colisiona con la relación course(), columnas score/writing_score/exam_score/attendance duplican al sistema real de grades/attendance, y columna "level" (enum básico/intermedio/avanzado) no coincide semánticamente con el "nivel" que la interfaz intenta mostrar.	INCONSISTENTE	Ver hallazgos 3, 4 y 5
courses	2 registros. Modelo Course. Incluye la columna nueva nivel_educativo, verificada correcta.	EN USO	
parallels	2 registros. Modelo Parallel. Sin observaciones.	EN USO	
subjects	1 registro. Modelo Subject. Sin observaciones.	EN USO	

Tabla	Qué se encontró (con evidencia)	Estado	Ref.
teaching_assignments	0 registros. Modelo TeachingAssignment. La tabla existe y el código la usa activamente (GradeController::hasTeachingAssignment), pero está vacía: bloquea que cualquier profesor califique en el estado actual de datos.	INCONSISTENTE	CRÍTICO — hallazgo 1
grades	20 registros. Modelo Grade. Núcleo del cuaderno pedagógico, en uso activo y verificado.	EN USO	
grade_history	0 registros. Modelo GradeHistory. Tabla y código correctos; vacía porque aún no se ha editado ninguna nota (comportamiento esperado, no es un error).	EN USO	Vacía por diseño, no por bug
attendance	140 registros. Modelo Attendance. En uso activo.	EN USO	
student_family_members	20 registros. Modelo StudentFamilyMember. Es el mecanismo REAL y vigente para vincular tutores/familiares a estudiantes.	EN USO	
activities	0 registros. Modelo Activity. Sin ninguna referencia real en controladores, vistas, rutas ni seeders (confirmado por grep exhaustivo); solo se referencia a sí mismo desde otros modelos del mismo subsistema huérfano.	SIN USO	Hallazgo 2 (subsistema huérfano)
activity_sessions	0 registros. Modelo ActivitySession. Mismo subsistema huérfano que "activities".	SIN USO	Hallazgo 2
classes	0 registros. Modelo SchoolClass (tabla renombrada a "classes"). Mismo subsistema huérfano.	SIN USO	Hallazgo 2
class_student	0 registros. Tabla pivote de SchoolClass ↔ Student. Mismo subsistema huérfano.	SIN USO	Hallazgo 2
groups	0 registros. Modelo Group. Mismo subsistema huérfano.	SIN USO	Hallazgo 2
group_student	0 registros. Tabla pivote de Group ↔ Student. Mismo subsistema huérfano.	SIN USO	Hallazgo 2
interactions	0 registros. Modelo Interaction (question/answer/is_correct/response_time: parece un diseño anterior para registrar interacciones de quiz). Mismo subsistema huérfano.	SIN USO	Hallazgo 2
results	0 registros. Modelo Result. Mismo subsistema huérfano.	SIN USO	Hallazgo 2
robot_alimentos	0 registros. Sin modelo Eloquent propio. Referenciada solo como valor de texto en un array PHP (nombre de categoría → nombre de tabla) en GradeController y RobotActivityController; nunca se ejecuta ninguna consulta real contra esta tabla (0 resultados de "DB::table(" en toda la carpeta app/).	SIN USO	Hallazgo 3
robot_colores	0 registros. Mismo caso que robot_alimentos.	SIN USO	Hallazgo 3
robot_casa_items	0 registros. Mismo caso que robot_alimentos.	SIN USO	Hallazgo 3

Tabla	Qué se encontró (con evidencia)	Estado	Ref.
robot_naturaleza_items	0 registros. Mismo caso que robot_alimentos.	SIN USO	Hallazgo 3
robot_familia_items	0 registros. Mismo caso que robot_alimentos.	SIN USO	Hallazgo 3
robot_herramientas_items	0 registros. Mismo caso que robot_alimentos.	SIN USO	Hallazgo 3
robot_lugares_items	0 registros. Mismo caso que robot_alimentos.	SIN USO	Hallazgo 3
(tabla "tutors")	NO EXISTE en la base de datos. El modelo Tutor.php y el método Student::tutors() la referencian activamente en el código como si existiera.	INCONSISTENTE	Hallazgo 4 — referencia a tabla inexistente
(tabla "student_tutor")	NO EXISTE en la base de datos. Es el pivote que Tutor::students() y Student::tutors() esperan encontrar; tampoco existe.	INCONSISTENTE	Hallazgo 4
personal_access_tokens	1 registro (token del robot). Tabla estándar de Laravel Sanctum, en uso normal.	EN USO	Framework
sessions	33 registros. Tabla estándar de Laravel (SESSION_DRIVER=database), en uso normal.	EN USO	Framework
cache / cache_locks	4 y 0 registros. Tablas estándar de Laravel (CACHE_STORE=database), en uso normal.	EN USO	Framework
jobs / job_batches / failed_jobs	0 registros cada una. Tablas estándar de colas de Laravel (QUEUE_CONNECTION=database); vacías porque el proyecto no ha encolado trabajos aún, no es un error.	EN USO	Framework, vacías por diseño
password_reset_tokens	0 registros. Tabla estándar de Laravel para recuperación de contraseña; vacía porque esa función no está implementada en la interfaz (no hay enlace "¿Olvidaste tu contraseña?" funcional, ya reportado en auditorías anteriores).	SIN USO	La función de recuperación de contraseña no está implementada en la UI
migrations	37 registros. Tabla interna de control de Laravel; coincide con las 37 migraciones reales del proyecto.	EN USO	Framework

3. HALLAZGOS DETALLADOS

Hallazgo 1 — teaching_assignments vacía bloquea la calificación (CRÍTICO)

Evidencia: SELECT COUNT(*) FROM teaching_assignments → 0. app/Http/Controllers/GradeController.php:307 y 545-556 exigen un registro activo ahí, incluso cuando el alumno ya tiene profesor asignado directamente.

Impacto: Ningún profesor puede guardar notas, asistencia ni actividades del robot con los datos reales actuales.

Ya reportado en: Informe_YURA_Auditoria_Pruebas.docx (ronda 2), sección 10, error 1. Este informe lo re-confirma desde la perspectiva específica de la base de datos.

Hallazgo 2 — Subsistema completo sin ninguna conexión real al sistema (8 tablas, 6 modelos)

Tablas: activities, activity_sessions, classes, class_student, groups, group_student, interactions, results (todas con 0 registros).

Modelos: Activity, ActivitySession, SchoolClass, Group, Interaction, Result.

Evidencia: grep -rn "Models\\Activity;|Models\\ActivitySession|Models\\Group;|Models\\Interaction|Models\\Result;|Models\\SchoolClass" en app/, resources/, routes/, database/seeders/ y tests/ → CERO resultados. Las únicas referencias a estas clases están dentro de sus propias relaciones mutuas (ej. ActivitySession::class es usado solo por Activity.php, SchoolClass.php).

Interpretación: Parece un diseño anterior para un sistema de "clases/grupos/interacciones/resultados" que fue reemplazado por el diseño actual (Grade con is_robot_activity y robot_activity_id), pero las tablas y modelos nunca se limpiaron.

Recomendación: Confirmar con el equipo si este subsistema tiene planes futuros; si no, eliminar los 8 modelos/tablas para reducir confusión y superficie de mantenimiento.

Hallazgo 3 — Las 7 tablas robot_* existen y tienen seeder, pero nunca se consultan

Evidencia: grep -rn "DB::table(" app/ → CERO resultados en todo el proyecto. GradeController.php:443-449 y RobotActivityController.php:26-32 solo usan los nombres de tabla como valores de texto dentro de un array PHP (categoría → nombre de tabla), nunca ejecutan una consulta real contra esas tablas. database/seeders/RobotCategorySeeder.php sí tiene datos definidos para sembrarlas (5 palabras por categoría), pero las 7 tablas están en 0 registros en la base real, por lo que ese seeder no se ejecutó sobre esta base de datos.

Impacto: Si en el futuro alguien espera que el sistema valide o liste vocabulario real desde estas tablas, hoy no lo hace: el nombre de la tabla es una etiqueta muerta.

Recomendación: Decidir si el vocabulario del robot debe vivir en estas tablas (y entonces sí conectarlas con consultas reales y correr el seeder) o si ya no son necesarias.

Hallazgo 4 — El modelo Tutor y Student::tutors() referencian tablas que no existen

Evidencia: Schema::hasColumn/hasTable confirmado por ausencia en SHOW TABLES: ni 'tutors' ni 'student_tutor' existen en la base de datos. app/Models/Tutor.php (modelo completo) y app/Models/Student.php:58-61 (método tutors()) siguen definiendo relaciones hacia esas tablas.

Reproducción: App\Models\Tutor::count() o $student->tutors()->get() lanzarían SQLSTATE[42S02]: Base table or view not found.

Nota: El mecanismo real y vigente para tutores/familiares es la tabla student_family_members (20 registros, en uso activo) — Tutor.php es código completamente huérfano de un diseño anterior.

Recomendación: Eliminar app/Models/Tutor.php y el método Student::tutors()/studentLinks(), ya que apuntan a tablas inexistentes y producirían una excepción si alguien los invoca.

Hallazgo 5 — students.course colisiona con la relación course(): bug real confirmado en vivo

Evidencia: Se ejecutó en tinker: $s = Student::with('course')->first(); $s->course. Resultado real: tipo string, valor "Sexto de Primaria" (el texto plano de la columna), AUNQUE la relación course() ya estaba cargada correctamente en $s->getRelations()['course'] como un objeto Course real. Eloquent siempre prioriza el atributo de columna sobre la relación cuando ambos tienen el mismo nombre.

Impacto reproducido: app/Http/Controllers/StudentController.php:71 ejecuta $student->course?->name para agrupar estudiantes. Se probó en vivo: PHP emite la advertencia real 'Attempt to read property "name" on string' y el resultado es

NULL, por lo que la variable $studentsByCourseAndParallel agrupa TODOS los estudiantes bajo la etiqueta 'Unassigned course', sin importar su curso real.

¿Se ve en pantalla?: No: se confirmó por grep que $studentsByCourseAndParallel se calcula en el controlador (línea 69) pero ninguna vista la usa actualmente. El bug es real y genera una advertencia de PHP en cada carga de /students, pero hoy no tiene impacto visible para el usuario final — sí lo tendría si algún día se usa esa variable.

Confirmación en el propio código: La migración database/migrations/2026_07_16_155353_drop_legacy_fields_from_students_table.php es un placeholder documentado que dice textualmente: "El campo course es un campo legado desnormalizado usado en toda la aplicación... puede hacerse en una futura iteración." El propio equipo ya sabía que este campo es un duplicado legado, pero nunca se corrigió la colisión de nombre con la relación.

Recomendación: Renombrar la relación a courseRelation() o eliminar la columna course en favor de course_id, actualizando los archivos que el propio comentario de la migración ya identifica (StudentController, DatabaseSeeder, tests, StudentsExport, reports/students_pdf.blade.php).

Hallazgo 6 — Migración "Ran" cuya columna ya no existe, sin ninguna migración que la elimine

Evidencia: database/migrations/2026_05_26_000001_add_estado_aprobacion_to_estudiantes.php está marcada como 'Ran' en la tabla migrations, y agrega la columna estudiantes.estado_aprobacion. Sin embargo, Schema::hasColumn('students','estado_aprobacion') devuelve NO en la base real.

Búsqueda de la causa: Se revisó la única migración con potencial de haberla eliminado (drop_legacy_fields_from_students_table.php), pero es un placeholder vacío (no-op) según su propio comentario. La migración de normalización (normalize_student_and_user_columns_to_english.php) solo contiene renameColumn(), ninguno sobre estado_aprobacion. NINGÚN archivo de migración del repositorio elimina explícitamente esta columna.

Interpretación: La columna fue eliminada por fuera del sistema de migraciones (por ejemplo, directamente en MySQL Workbench o con una consulta manual), o en algún momento se recreó la base de datos sin ese estado intermedio. Esto es exactamente el tipo de desincronización entre "lo documentado en las migraciones" y "lo que existe realmente" que se pidió auditar.

Recomendación: Si esta columna ya no es necesaria, crear una migración formal que la elimine (para que el historial de migraciones sea la fuente de verdad real); si aún se necesita, crear una migración que la vuelva a agregar.

Hallazgo 7 — students.level no coincide semánticamente con lo que la interfaz muestra como "nivel"

Evidencia: La columna level es un enum('básico','intermedio','avanzado') — un nivel de competencia, no un grado escolar. Se confirmó con una consulta real que los 20 estudiantes sembrados tienen el valor 'básico'. La vista students/index.blade.php muestra este valor en un badge junto al nombre del estudiante como si fuera su 'nivel' académico (6to, etc.), cuando en realidad siempre mostrará 'BÁSICO' para todos.

Impacto: Visual/de contenido, no rompe la aplicación: el badge simplemente no aporta información académica útil hoy, porque la columna que lee no fue diseñada para eso.

Recomendación: Aclarar con el equipo si "level" debe pasar a representar el grado escolar (y entonces poblarlo correctamente) o si el badge debe leer otro campo (por ejemplo, derivarlo de courses.name).

Hallazgo 8 — Columnas duplicadas de puntaje/asistencia en students, paralelas al sistema real

Evidencia: students tiene las columnas score, writing_score, exam_score, attendance (todas int, con valores por defecto), y siguen en $fillable y en las reglas de validación de StudentController (líneas 157-160). Sin embargo, el promedio real que se muestra en pantalla (promedio_real / promedio_acumulado) se calcula a partir de la relación grades() (tabla grades, 20 registros), NO de estas columnas.

Impacto: Quien edite manualmente el "puntaje" de un estudiante desde su ficha puede pensar que está afectando su promedio real, pero ese valor queda aislado y no se usa en ningún cálculo mostrado al usuario.

Recomendación: Retirar estos 4 campos del formulario de edición de estudiante si ya no deben usarse, o documentar claramente su propósito si se conservan a propósito.

Hallazgo 9 — users.role y users.rol: columnas duplicadas confirmadas a nivel de esquema real

Evidencia: SHOW FULL COLUMNS FROM users confirma ambas columnas presentes simultáneamente: role varchar(255) default 'padre' y rol varchar(255) default 'padre'. app/Models/User.php las mantiene sincronizadas mediante un método de compatibilidad (writeCompatAttribute).

Ya reportado en: Plan de Pruebas e Informe de Auditoría anteriores, como riesgo de inconsistencia role/rol. Este hallazgo lo confirma con evidencia exacta de columna real (tipo, default, nulabilidad).

Recomendación: Decidir una sola columna definitiva (probablemente role, en inglés, consistente con el resto de la normalización) y crear una migración que elimine la otra tras verificar que ningún código la lee directamente.

4. ¿EXISTE UNA SEGUNDA CONEXIÓN A SQLITE PARA OTRO SISTEMA?

Respuesta directa, con evidencia: NO existe ninguna conexión activa hacia una base de datos SQLite para un sistema distinto al de esta plataforma web.

Evidencia revisada

- config/database.php contiene las conexiones por defecto que trae cualquier proyecto Laravel (sqlite, mysql, pgsql, sqlsrv) — es el archivo de plantilla estándar del framework, no una configuración a medida para un segundo sistema.
- El archivo .env actual (el que realmente usa la aplicación en ejecución) define DB_CONNECTION=mysql y no menciona SQLite en ningún punto. Solo el archivo de ejemplo .env.example conserva sqlite como valor por defecto de fábrica de Laravel, nunca aplicado en este entorno real.
- grep -rn "DB::connection(" en app/, config/ y routes/ → CERO resultados: en ningún punto del código la aplicación abre una segunda conexión de base de datos explícita. Todo el sistema usa siempre la conexión "default" (hoy mysql).
- El archivo físico database/database.sqlite SÍ existe en disco (286 720 bytes, base SQLite 3.x válida, última modificación 2026-09-16 17:48h) — pero corresponde a la configuración anterior de este mismo proyecto, de antes de migrar a MySQL en esta misma sesión de trabajo. No hay ninguna variable de entorno ni código que la referencie hoy.
- El único lugar del repositorio que sí configura sqlite activamente es phpunit.xml, con DB_CONNECTION=sqlite y DB_DATABASE=:memory:. Esto es exclusivamente para los tests automatizados (php artisan test): Laravel crea una base SQLite en memoria, vacía, que se destruye al terminar cada ejecución de pruebas. No es un archivo persistente ni se comparte con ningún otro sistema.

Conclusión sobre superposición o conflicto

No hay superposición: la aplicación en ejecución (web, artisan, tinker) usa exclusivamente MySQL (db_yura_platform). Los tests automatizados usan una base SQLite en memoria completamente aislada y efímera. El archivo database/database.sqlite es un remanente inerte que no participa en ninguna operación actual del sistema. No se detectó ningún "otro sistema" real que dependa de una base SQLite separada; la pregunta parece originarse en la existencia de ese archivo legado, ya explicada.

- Recomendación: eliminar database/database.sqlite si ya no se planea volver a SQLite, para evitar confusión futura, o documentar explícitamente que es un respaldo de la configuración anterior.

5. CONCLUSIONES

La base de datos MySQL real (db_yura_platform) está correctamente conectada y en uso por la aplicación: las 37 migraciones están aplicadas y la mayoría de las tablas de negocio (11 de 19) están activamente conectadas al código y en uso real, con datos consistentes.

Sin embargo, la auditoría confirma con evidencia directa —no con suposiciones— que existe una cantidad significativa de deuda técnica en forma de tablas y modelos sin ninguna conexión real al sistema (10 tablas de negocio sin uso, agrupadas principalmente en un subsistema completo de 8 tablas/6 modelos y en el conjunto de 7 tablas de vocabulario del robot), y 4 inconsistencias reales entre lo que el código asume y lo que existe en la base de datos: dos tablas referenciadas por código que no existen (tutors, student_tutor), un bug de colisión de nombre entre una columna y una relación Eloquent (students.course) confirmado en vivo con una advertencia real de PHP, y una columna de migración "Ran" que ya no existe sin ninguna migración que documente su eliminación (estudiantes.estado_aprobacion).

Se confirma también, con evidencia directa, que no existe ninguna segunda base de datos SQLite activa para un sistema distinto: el archivo SQLite presente es un remanente inerte de la configuración anterior de este mismo proyecto, y el único uso activo de SQLite es la base en memoria, efímera, que Laravel crea automáticamente solo durante los tests automatizados.

Ninguno de los hallazgos de este informe se basa en la documentación o en el código por sí solos: cada uno fue verificado contra el estado real de la base de datos MySQL en ejecución, con consultas SQL reales, búsquedas exhaustivas en el código fuente, y reproducción en vivo cuando fue posible.